CS614 - Endsem Project Review

Aditi Khandelia **Arush Upadhyaya** **Kushagra Srivastava** **Sankalp Mittal** **Vidhi Jain**

April 21, 2026

Project Overview

Iterative pre-copy live migration transfers memory pages from a running process to a destination host while the process continues to execute, repeating the transfer for pages that were dirtied during the previous round until the remaining working set is small enough to complete migration with a brief final pause. The efficiency of this scheme depends entirely on the ability to identify, at the end of each round, *exactly* which pages were written. For CPU memory, the Linux kernel provides this capability through soft-dirty bits embedded in page-table entries, readable via `/proc/PID/pagemap`. No analogous mechanism exists for GPU-managed memory. NVIDIA's Unified Virtual Memory (UVM) subsystem, which underpins `cudaMallocManaged` and handles demand-paging between host and device, exposes no interface for querying which pages a process has modified. Consequently, any migration or checkpointing system must conservatively copy the *entire* GPU memory footprint on every iteration, rendering iterative pre-copy migration of large GPU workloads impractical.

This project addresses that gap by implementing **per-process dirty page tracking** directly within the open-source NVIDIA UVM kernel module. UVM already intercepts every GPU page fault to manage demand-paging; we augment the write-fault servicing path to record each written page in an in-kernel data structure, without introducing any additional hardware overhead. A `procfs`-based interface allows a privileged user to start and stop tracking epochs, perform atomic cutover snapshots, pause and resume recording, and retrieve the set of pages written by a given process with **page-level granularity** and a per-page **nanosecond-resolution timestamp** suitable for ordering events across migration rounds.

The core mechanism exploits the GPU's demand-paging fault path. At the start of each tracking epoch, write permissions are *downgraded* from all currently mapped GPU pages, leaving them readable but not writable. Any subsequent GPU write generates a replayable fault; the driver's fault-servicing path records the faulting page and timestamp before restoring write permission and replaying the instruction. Pages that are only read never fault and are not recorded, eliminating the overhead of tracking read-only accesses. Formal determinism guarantees ensure that no write escapes recording and that the boundary between consecutive migration rounds is precisely defined through a barrier-protected atomic table swap.

Group Details

Following are the details of the group members:

Name	Roll No.	Email ID
Aditi Khandelia	220061	aditikh22@iitk.ac.in
Arush Upadhyaya	220213	arushu22@iitk.ac.in
Kushagra Srivastava	220573	skushagra22@iitk.ac.in
Sankalp Mittal	220963	sankalpm22@iitk.ac.in
Vidhi Jain	221191	vidhijain22@iitk.ac.in

Implementation

End-to-end Workflow

Modified Driver Files

All driver modifications reside under `kernel-open/nvidia-uvm/`. The files containing functionality added or significantly modified by us are:

1. `uvm_common.h` / `uvm_common.c` - The primary implementation file. Defines `struct dirty_page_info` (page number, timestamp) and the three global dirty-set tables (`g_dirty_ds`, `g_dirty_ds_snapshot`, `g_dirty_ds_cumulative`). Implements table lifecycle (`uvm_dirty_page_table_init`, `uvm_dirty_page_table_destroy`), fault recording (`uvm_dirty_page_table_record`), snapshot management (`uvm_dirty_new_snapshot`, `uvm_dirty_merge_snapshot_into_cumulative_locked`), and all eight `procfs` handler implementations including `uvm_dirty_procfs_init`. Exports function pointers (`uvm_dirty_invalidate_fn`, `uvm_dirty_activate_now_fn`, `uvm_dirty_barrier_end_fn`, `uvm_dirty_query_barrier_begin_fn`, `uvm_dirty_query_barrier_end_fn`) that are registered from `uvm_va_space.c` at module init.
2. `uvm_dirty_ds.h` - Defines the `uvm_dirty_ds_ops` interface (init, insert, lookup, for_each_in_range, destroy), the `uvm_dirty_ds` struct holding ops pointer, backend-private data, and statistics, and inline timing wrappers for all operations.
3. `uvm_dirty_ds_xarray.c` - xarray backend using atomic `xa_cmpxchg` for first-write-wins insert.
4. `uvm_dirty_ds_vector.c` - Sorted dynamic array backend with binary-search insert and lookup.
5. `uvm_dirty_ds_vector_unsorted.c` - Unsorted append-order vector with spinlock, preserving fault-arrival order.
6. `uvm_dirty_ds_linked_list.c` - Singly-linked list with sentinel node.
7. `uvm_dirty_ds_bitmap.c` - Flat bit array covering 2^{38} pages (32 MB fixed allocation); O(1) operations; timestamps not stored.
8. `uvm_dirty_ds_nested_bitmap.c` - Two-level hierarchical bitmap for sparse page sets.
9. `uvm_dirty_ds_chunked.c` - Chunked vector with a background kthread sorter for deferred ordering.
10. `uvm_gpu_replayable_faults.c` - The write-fault hook. After standard fault servicing, checks `uvm_dirty_tracking_active()` and `service_access_type >= UVM_FAULT_ACCESS_TYPE_WRITE`, then calls `uvm_dirty_page_table_record` with the faulting page number and `ktime_get_ns()` timestamp.
11. `uvm_va_space.c` - Implements the barrier infrastructure. `uvm_dirty_downgrade_all_permissions` walks all va-ranges and va-blocks for the owner's va_space, calling `uvm_va_block_revoke_prot_mask` with `UVM_PROT_READ_WRITE` to strip write access from all GPU PTEs, followed by `uvm_tracker_wait` to confirm GPU processing. `uvm_dirty_query_barrier_begin` and `uvm_dirty_query_barrier_end` implement the cutover barrier by locking all parent GPU replayable-fault ISRs and spinning until the fault buffer is empty. `uvm_va_space_dirty_init` registers all five function pointers at module init.
12. `uvm_va_block.c` - `compute_new_permission` modified to force `READ_ONLY` on read faults when tracking is active for the faulting va_space's owner, preventing read faults from obtaining write mappings that would bypass recording.
13. `uvm_migrate.c` - `block_migrate_map_unmapped_pages` modified to map GPU-migrated pages as `READ_ONLY` when tracking is active, closing the `cudaMemPrefetchAsync` bypass.
14. `uvm.c` - Top-level integration; calls `uvm_dirty_procfs_init` and `uvm_va_space_dirty_init` during module initialisation.

Procfs Interface

Eight procfs entries are created under `/proc/driver/nvidia-uvm/`:

1. `dirty_tracking_start` (write) - Input `"<pid> <delta|cumulative>"`. Initialises the live table, downgrades all GPU write permissions, and sets `active = true`.
2. `dirty_tracking_stop` (write) - Input `"<pid>"`. Destroys all tracking state (live, snapshot, cumulative tables) and flushes accumulated statistics to `/tmp/uvm_dirty_ds_stats.txt`.
3. `dirty_tracking_pause` (write) - Input `"<pid>"`. Soft gate flip: sets `active = false`. In-flight fault handlers may still complete a record insertion.
4. `dirty_tracking_resume` (write) - Input `"<pid>"`. Re-runs the full downgrade and activation sequence, re-establishing the deterministic write-fault frontier.

5. `dirty_tracking_query_cutover` (write) - Input `"<pid>"`. Acquires the query barrier, atomically swaps live table to snapshot and installs a fresh live table, increments `g_dirty_epoch`, sets `g_snapshot_pending = true`, releases barrier. Returns `EBUSY` if a snapshot is already pending.
6. `dirty_tracking_query_dump` (read) - Requires `g_snapshot_pending = true`. In `delta` mode emits the snapshot pages and destroys it. In `cumulative` mode merges snapshot into `g_dirty_ds_cumulative` (first-write-wins) then emits the cumulative union. Output format: one line per page, `0x`*`addr timestamp_ns`*.
7. `dirty_ds_stats_toggle` (write) - Input `"enable"` or `"disable"`. Enabling resets all counters for a clean measurement window.
8. `dirty_ds_stats` (read) - Emits per-operation counts and total nanoseconds for init, insert, lookup, for-each, destroy, insert lock-wait, lookup lock-wait, and the three callback operations (invalidate, activate, barrier-end).

Design Choices

Determinism Guarantees

Data Structures

Global state Three `uvm_dirty_ds` instances hold the tracking data:

1. `g_dirty_ds` - live table; fault handler inserts here.
2. `g_dirty_ds_snapshot` - frozen copy created by cutover; exists only between a cutover and the subsequent dump.
3. `g_dirty_ds_cumulative` - session-union table used in cumulative mode; merge-accumulates snapshot contents at each dump.

Backend abstraction All three tables share the `uvm_dirty_ds_ops` interface (`uvm_dirty_ds.h`), which provides init, insert, lookup, for_each_in_range, and destroy. Inline wrappers measure operation latency when statistics are enabled. The active backend is selected at compile time by setting the `.ops` pointer in the static `g_dirty_ds` initialiser.

Locking

A read-write semaphore (`uvm_dirty_ds_rwsem`) protects all three tables. Fault-handler insertions and lookups take the read side concurrently. Lifecycle operations (init, destroy, cutover swap) take the write side exclusively, blocking new insertions only during the brief pointer swap. A separate mutex (`uvm_dirty_lifecycle_lock`) serialises the procfs lifecycle handlers (start, stop, pause, resume, cutover) against each other.

Testing and Analysis

Future Work

Source Code

Development Environment

Development and testing are conducted on a dedicated virtual machine (`dirtyuvm`) running:

- Linux kernel version 6.8 (custom build)
- Modified NVIDIA UVM driver version 580.65.06 (loaded as a kernel module)

Repository

The full source code is hosted at:

`https://github.com/aleph-5/open-gpu-kernel-modules`

This is a fork of NVIDIA's open-source GPU kernel modules, with our modifications applied on top.

Instructions for Run

Declaration

We declare that the work presented in this report is our own, except where otherwise acknowledged. We have also read and understood the University's policy on plagiarism and academic integrity, and we confirm that this report complies with those standards.